

Generator of random numbers

The goal of these two exercise sessions is to implement some generators of random numbers, and investigate their behavior in terms of accuracy and convergence.

1 Implementing random generators

A large list of generators is proposed, their implementation can be selected with respect to your own interest or previous experiences.

1.1 Random generators based on Monte Carlo method

Linear congruential generator for uniform distribution on $]0; 1[$

1. Implement a linear congruential generator (LCG) of uniform random numbers distributed on $]0; 1[$.

The implementation of the linear congruential generator is as follows:

$$\begin{cases} u_0 = \text{seed} \\ u_i = (ax_{i-1} + c) \bmod m, \quad \forall i \in [1; N] \end{cases} \quad (1)$$

where a is called the multiplier, c is called the increment and m is called the modulus. More details available in *Random Number Generation and Monte Carlo Methods*, James E. Gentle, p.11-14.

Multiplicative congruential generator for uniform distribution on $]0; 1[$

2. How to change your previous code to implement a multiplicative congruential generator with for example $m = 2^{13} - 1$ and $a = 17$.
3. Modify your program to implement a matrix congruential generator:

$$\begin{bmatrix} x_{1i} \\ x_{2i} \end{bmatrix} \equiv \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix} \begin{bmatrix} x_{1,i-1} \\ x_{2,i-1} \end{bmatrix} \bmod m$$

with $m = 2^{13} - 1$, $a_{11} = 17$, $a_{22} = 85$ and a_{12} and a_{21} variable. Choose some values for a_{12} and a_{22} between 0 and 17.

1.2 Generators based on a Latin Hypercube sampling

4. Implement a Latin hypercube sampling of Gaussian distributed independent random variables, or independent random vectors.

Remark: These samples could also be provided by the Matlab functions: `lhsdesign` and `lhsnorm`.

5. How to change your algorithm to provide an orthogonal Latin hypercube sampling of Gaussian distributed independent random vectors?

1.3 Generators based on Quasi Random sequences

Sobol sequence

6. We could implement a Sobol generator for uniform distribution of one-dimensional vector on $]0; 1[$, or for two-dimensional random vectors on $]0; 1]^2$. But, for this session, we will use the Sobol samples which are provided by the Matlab function `sobolset`.

Halton sequence

7. We could implement a Halton generator for uniform distribution of one-dimensional vector on $[0; 1]$, or for two-dimensional random vectors on $[0; 1]^2$. But, for this session, we will use the Halton samples which are provided by the Matlab function `haltonset`.

1.4 Generators for other distributions

Based on the inverse CDF method

8. Prove that if X is a random variable with an absolutely continuous distribution function F_X , the random variable $F_X(X)$ has a uniform distribution on $]0; 1[$.
9. Implement a generator of a set of uniform random numbers on $]a; b[$ from a uniform set of data.
10. Implement a generator of a set of exponential random numbers on $[0; +\infty[$ from a uniform set of data.

Using Matlab

11. Generate a set of Gaussian random numbers using the function `normrnd`.
12. Generate a set of log-normal, and Weibull random numbers.

2 Investigate the repeatability of the generators

13. How to provide a repeatability of the sample of random variables, using the different generators previously implemented or used?
14. At the contrary, how to ensure to get different random sets?

3 Investigate the accuracy of the generators

In order to investigate their accuracy, we will not propose here to analyze mathematically the algorithm used by the generators. We will only observe some streams of numbers produced by the different generators, and investigate how their properties are closed to the expected properties.

15. To which properties can we refer to investigate this accuracy?
16. For one or several of these properties, investigate the accuracy of one of the previous generators. For example, investigate the lattice structure or the correlation of the pairs (x_n, x_{n+1}) for a multiplicative congruential method depending on the value of the multiplier.

4 Investigate the convergence of the generators

17. To investigate the convergence of a generator, we need to refer to a quantity of interest. Which quantities of interest could you consider to investigate the convergence of a generator?
18. Choose one generator, and investigate the convergence behavior of this generator with respect to some quantities of interest. For example, investigate the lattice structure of a two-dimensional Sobol generator.
19. Does the convergence behavior satisfy the results found in the literature?